

Chapter 40 — Bitmask Dynamic Programming

0. What You Will Learn in This Chapter

Bitmask Dynamic Programming is used when the state depends on a subset of items.

A bitmask is an integer whose binary representation tells which items are selected, visited, used, or assigned.

Bitmask DP is common when:

- n is small, usually around 10 to 20
- you need to try subsets
- you need to visit all nodes
- you need to assign tasks to workers
- you need to form groups
- you need to remember which elements were used
- you need shortest path over subsets
- you need traveling-salesman-style DP

By the end of this chapter, you should understand:

- What a bitmask is
- How to represent subsets with bits
- How to set, clear, and test bits
- How to iterate over masks
- How to iterate over submasks
- How to define $dp[mask]$
- How to define $dp[mask][last]$
- How to solve assignment DP
- How to solve TSP-style DP
- How to combine BFS with bitmask states
- How to avoid exponential-state mistakes

Core problems covered in this chapter:

1. Subsets with Bitmask
 2. Can I Win
 3. Partition to K Equal Sum Subsets
 4. Matchsticks to Square
 5. Beautiful Arrangement
 6. Minimum XOR Sum of Two Arrays
 7. Shortest Path Visiting All Nodes
 8. Traveling Salesman Problem
 9. Number of Ways to Wear Different Hats
 10. Smallest Sufficient Team
 11. Max Students Taking Exam
 12. Minimum Incompatibility
 13. Parallel Courses II
 14. Stickers to Spell Word
 15. Campus Bikes II
-

1. What Is a Bitmask?

A bitmask is an integer used to represent a set.
Each bit represents whether an item is included.
Example with 4 items:

items: 0 1 2 3
mask: 1 0 1 1

Binary:

1011

Means:

item 0 included
item 1 included
item 2 not included
item 3 included

In decimal:

$1011_2 = 11$

So mask 11 represents subset {0,1,3} if bit 0 is the rightmost bit.

2. Why Bitmask DP Exists

When n is small, we can represent all subsets.
Number of subsets:

2^n

For $n = 20$:

about 1,048,576 subsets

This can be feasible.
Instead of using a set or vector to track chosen items, we use an integer.
This makes subset state compact and fast.

3. Bit Operations You Must Know

3.1 Check If Bit i Is Set

```
if (mask & (1 << i)) {  
    // item i is included  
}
```

3.2 Set Bit i

```
mask | (1 << i)
```

3.3 Clear Bit i

```
mask & ~(1 << i)
```

3.4 Toggle Bit i

```
mask ^ (1 << i)
```

3.5 Count Set Bits

```
__builtin_popcount(mask)
```

For long long:

```
__builtin_popcountll(mask)
```

3.6 Full Mask

For n items:

```
int fullMask = (1 << n) - 1;
```

Example:

```
n = 4
```

```
fullMask = 11112
```

4. Bitmask DP State Types

4.1 dp[mask]

Used when the subset alone defines the state.

Example:

```
dp[mask] = minimum cost to assign first popcount(mask) workers to selected jobs
```

4.2 dp[mask][last]

Used when you need to know the subset and the last chosen item.

Example:

```
dp[mask][last] = shortest path visiting nodes in mask and ending at last
```

4.3 visited[node][mask]

Used in BFS over graph state.

Example:

```
Shortest Path Visiting All Nodes
```

4.4 memo[mask]

Used in recursive search where used elements are represented by mask.

Example:

```
Can I Win
```

5. When to Use Bitmask DP

Use bitmask DP when:

5.1 n Is Small

Usually:

$n \leq 20$

Sometimes:

$n \leq 15$

for $dp[mask][last]$.

5.2 Need Track Used Items

Examples:

- assignment
- permutation
- grouping
- matching

5.3 Need Visit All Nodes

Examples:

- TSP
- Shortest Path Visiting All Nodes

5.4 Need Try Subsets

Examples:

- Smallest Sufficient Team
- Minimum Incompatibility

5.5 Need Group Formation

Examples:

- Partition to K Equal Sum Subsets
 - Matchsticks to Square
-

6. When NOT to Use Bitmask DP

Do not use bitmask DP when:

6.1 n Is Too Large

If $n = 100$, 2^n is impossible.

6.2 State Is Continuous or Numeric Large

Use normal DP, greedy, graph, or math.

6.3 Need Range Queries

Use segment tree/Fenwick tree.

6.4 Need Simple Backtracking Output

If only generating all subsets, bitmask enumeration may be enough, not DP.

6.5 Graph Has Large Node Count

Bitmask graph DP is only for small graphs.

7. Mental Models

7.1 Mask as Memory

The mask remembers what has already been used.

7.2 Build Subsets

Start with empty mask:

```
0000
```

Add items one by one until full mask:

```
1111
```

7.3 Remove One Item

To compute $dp[mask]$, try removing the last chosen item.

```
previousMask = mask without item i
```

7.4 BFS State

In graph problems, state is not just node.

It is:

```
current node + visited nodes mask
```

8. Pattern Recognition Checklist

Ask:

16. Is n small, around 10 to 20?
17. Do I need to track which items are used?
18. Is the problem about subsets or permutations?
19. Do I need to visit all nodes?
20. Is there an assignment between two groups?
21. Does state need subset + last item?
22. Can the chosen set be represented by bits?
23. Is brute force factorial but DP can reduce to $n * 2^n$?
24. Do I need memoized recursion with used elements?
25. Does the problem mention all subsets, teams, hats, jobs, or courses?

9. C++ Tools Needed

9.1 Create Full Mask

```
int fullMask = (1 << n) - 1;
```

9.2 DP Array

```
vector<int> dp(1 << n, INF);
```

9.3 2D DP

```
vector<vector<int>> dp(1 << n, vector<int>(n, INF));
```

9.4 Memo Map

```
unordered_map<int, int> memo;
```

or:

```
vector<int> memo(1 << n, -1);
```

9.5 Queue for BFS State

```
queue<pair<int, int>> q; // node, mask
```

10. Core Templates

Template 1 — Iterate All Masks

```
for (int mask = 0; mask < (1 << n); mask++) {  
    // process subset mask  
}
```

Template 2 — Iterate Items in Mask

```
for (int i = 0; i < n; i++) {  
    if (mask & (1 << i)) {  
        // item i is included  
    }  
}
```

Template 3 — Iterate Items Not in Mask

```
for (int i = 0; i < n; i++) {  
    if ((mask & (1 << i)) == 0) {  
        int nextMask = mask | (1 << i);  
    }  
}
```

Template 4 — DP Over Subsets

```
vector<int> dp(1 << n, INF);  
dp[0] = 0;  
  
for (int mask = 0; mask < (1 << n); mask++) {  
    int used = __builtin_popcount(mask);  
  
    for (int i = 0; i < n; i++) {  
        if ((mask & (1 << i)) == 0) {  
            int nextMask = mask | (1 << i);
```

```

        dp[nextMask] = min(dp[nextMask],
                           dp[mask] + cost(used, i));
    }
}

```

Template 5 — TSP-Style DP

```

vector<vector<int>> dp(1 << n, vector<int>(n, INF));

dp[1 << start][start] = 0;

for (int mask = 0; mask < (1 << n); mask++) {
    for (int last = 0; last < n; last++) {
        if (dp[mask][last] == INF) {
            continue;
        }

        for (int next = 0; next < n; next++) {
            if ((mask & (1 << next)) == 0) {
                int nextMask = mask | (1 << next);

                dp[nextMask][next] = min(dp[nextMask][next],
                                          dp[mask][last] + dist[last][next]);
            }
        }
    }
}

```

Template 6 — Iterate Submasks

```

for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // sub is a non-empty submask of mask
}

```

Include empty submask separately if needed.

11. Major Bitmask DP Variations

Used items	dp[mask]	Assignment

Variation	State	Example
Used + last	dp[mask][last]	TSP
Graph BFS	visited[node][mask]	Shortest Path Visiting All Nodes
Recursive game	memo[mask]	Can I Win
Grouping	dp[mask] with sum mod target	Partition K Subsets
Skills/team	dp[skillMask]	Smallest Sufficient Team
Hats/items inverted	dp[personMask]	Hats
Grid row masks	dp[row][mask]	Max Students Exam

12. Full Problem Walkthrough 1 — Subsets With Bitmask

12.1 Problem Summary

Generate all subsets of an array.

12.2 Bitmask Idea

For array size n , each mask from:

0 to $2^n - 1$

represents one subset.

12.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        int n = nums.size();

        vector<vector<int>> result;

        for (int mask = 0; mask < (1 << n); mask++) {
            vector<int> subset;

            for (int i = 0; i < n; i++) {
                if (mask & (1 << i)) {
                    subset.push_back(nums[i]);
                }
            }
        }
    }
};
```

```

        }

        result.push_back(subset);
    }

    return result;
}
};

```

12.4 Complexity

Time:

$O(n * 2^n)$

Space:

$O(n * 2^n)$

for output.

13. Full Problem Walkthrough 2 — LeetCode 464: Can I Win

13.1 Problem Summary

Players take turns choosing numbers from 1 to maxChoosableInteger.

Each number can be chosen once.

Running total increases.

Player who reaches or exceeds desiredTotal wins.

Return whether first player can force a win.

13.2 How to Identify Bitmask DP

Need remember which numbers were already used.

maxChoosableInteger is small enough for bitmask.

13.3 State

memo[mask] = whether current player can force a win

mask represents used numbers.

13.4 Transition

Try choosing unused number i.

If choosing i reaches target, win.

Otherwise, if opponent loses after this choice, current wins.

13.5 C++ Code

```

class Solution {
public:
    int maxNum;
    int desired;

```

```

vector<int> memo;

bool dfs(int mask, int remaining) {
    if (memo[mask] != -1) {
        return memo[mask];
    }

    for (int i = 1; i <= maxNum; i++) {
        int bit = 1 << (i - 1);

        if ((mask & bit) == 0) {
            if (i >= remaining) {
                memo[mask] = 1;
                return true;
            }

            if (!dfs(mask | bit, remaining - i)) {
                memo[mask] = 1;
                return true;
            }
        }
    }

    memo[mask] = 0;
    return false;
}

bool canIWin(int maxChoosableInteger, int desiredTotal) {
    maxNum = maxChoosableInteger;
    desired = desiredTotal;

    if (desiredTotal <= 0) {
        return true;
    }

    int total = maxChoosableInteger * (maxChoosableInteger + 1) / 2;

    if (total < desiredTotal) {
        return false;
    }

    memo.assign(1 << maxChoosableInteger, -1);

```

```
        return dfs(0, desiredTotal);
    }
};
```

13.6 Complexity

Time:

$O(n * 2^n)$

Space:

$O(2^n)$

14. Full Problem Walkthrough 3 — LeetCode 698: Partition to K Equal Sum Subsets

14.1 Problem Summary

Partition nums into k subsets with equal sum.

14.2 How to Identify Bitmask DP

Need track which numbers are used.

n is small.

14.3 State

dp[mask] = current bucket sum modulo target, or -1 if unreachable

If dp[mask] != -1, mask is reachable.

14.4 C++ Code

```
class Solution {
public:
    bool canPartitionKSubsets(vector<int>& nums, int k) {
        int total = accumulate(nums.begin(), nums.end(), 0);

        if (total % k != 0) {
            return false;
        }

        int target = total / k;
        int n = nums.size();

        sort(nums.rbegin(), nums.rend());
```

```

    if (nums[0] > target) {
        return false;
    }

    int full = 1 << n;

    vector<int> dp(full, -1);
    dp[0] = 0;

    for (int mask = 0; mask < full; mask++) {
        if (dp[mask] == -1) {
            continue;
        }

        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) == 0) {
                if (dp[mask] + nums[i] <= target) {
                    int nextMask = mask | (1 << i);
                    dp[nextMask] = (dp[mask] + nums[i]) % target;
                }
            }
        }
    }

    return dp[full - 1] == 0;
}
};

```

14.5 Complexity

Time:

$O(n * 2^n)$

Space:

$O(2^n)$

15. Full Problem Walkthrough 4 — Matchsticks to Square

15.1 Problem Summary

Given matchsticks, determine if they can form a square. Each side must have equal length.

15.2 Relation to K Equal Sum Subsets

This is exactly:

Partition to 4 equal sum subsets

15.3 C++ Code

```
class Solution {
public:
    bool makesquare(vector<int>& matchsticks) {
        int total = accumulate(matchsticks.begin(), matchsticks.end(), 0);

        if (total % 4 != 0) {
            return false;
        }

        int target = total / 4;
        int n = matchsticks.size();

        sort(matchsticks.rbegin(), matchsticks.rend());

        if (matchsticks[0] > target) {
            return false;
        }

        vector<int> sides(4, 0);

        function<bool(int)> dfs = [&](int index) {
            if (index == n) {
                return sides[0] == target &&
                    sides[1] == target &&
                    sides[2] == target &&
                    sides[3] == target;
            }

            for (int side = 0; side < 4; side++) {
                if (sides[side] + matchsticks[index] <= target) {
                    sides[side] += matchsticks[index];

                    if (dfs(index + 1)) {
                        return true;
                    }

                    sides[side] -= matchsticks[index];
                }
            }
        };
    }
};
```

```

        if (sides[side] == 0) {
            break;
        }
    }

    return false;
};

return dfs(0);
}
};

```

15.4 Note

This version uses backtracking.

A bitmask DP version is similar to Partition to K Equal Sum Subsets.

16. Full Problem Walkthrough 5 — LeetCode 526: Beautiful Arrangement

16.1 Problem Summary

Count permutations of numbers 1 to n such that for position pos:

```
num % pos == 0 or pos % num == 0
```

16.2 How to Identify Bitmask DP

Need assign unused numbers to positions.

16.3 State

dp[mask] = number of ways to fill first popcount(mask) positions

16.4 C++ Code

```

class Solution {
public:
    int countArrangement(int n) {
        vector<int> dp(1 << n, 0);

        dp[0] = 1;

        for (int mask = 0; mask < (1 << n); mask++) {
            int pos = __builtin_popcount(mask) + 1;

```

```

    for (int num = 1; num <= n; num++) {
        int bit = 1 << (num - 1);

        if ((mask & bit) == 0) {
            if (num % pos == 0 || pos % num == 0) {
                dp[mask | bit] += dp[mask];
            }
        }
    }

    return dp[(1 << n) - 1];
};

```

16.5 Complexity

Time:

$O(n * 2^n)$

Space:

$O(2^n)$

17. Full Problem Walkthrough 6 — LeetCode 1879: Minimum XOR Sum of Two Arrays

17.1 Problem Summary

Pair each element of `nums1` with one unused element of `nums2`.
Minimize sum of XORs.

17.2 How to Identify Assignment DP

Need assign `nums2` elements to `nums1` positions.
`n` is small.

17.3 State

`dp[mask]` = minimum XOR sum after assigning first `popcount(mask)` `nums1` elements
to selected `nums2` elements

17.4 C++ Code

```

class Solution {
public:
    int minimumXORSum(vector<int>& nums1, vector<int>& nums2) {

```

```

int n = nums1.size();

const int INF = 1e9;

vector<int> dp(1 << n, INF);
dp[0] = 0;

for (int mask = 0; mask < (1 << n); mask++) {
    int i = __builtin_popcount(mask);

    for (int j = 0; j < n; j++) {
        if ((mask & (1 << j)) == 0) {
            int nextMask = mask | (1 << j);

            dp[nextMask] = min(dp[nextMask],
                               dp[mask] + (nums1[i] ^ nums2[j]));
        }
    }
}

return dp[(1 << n) - 1];
};

```

17.5 Complexity

Time:

$O(n * 2^n)$

Space:

$O(2^n)$

18. Full Problem Walkthrough 7 — LeetCode 847: Shortest Path Visiting All Nodes

18.1 Problem Summary

Given connected undirected graph.

Return shortest path length that visits every node.

Can start and end anywhere.

Can revisit nodes.

18.2 How to Identify Bitmask BFS

Need track:

current node
visited set

This is BFS over states.

18.3 State

(node, mask)

where mask represents visited nodes.

18.4 C++ Code

```
class Solution {
public:
    int shortestPathLength(vector<vector<int>>& graph) {
        int n = graph.size();

        int fullMask = (1 << n) - 1;

        queue<pair<int, int>> q;
        vector<vector<bool>> visited(n, vector<bool>(1 << n, false));

        for (int node = 0; node < n; node++) {
            int mask = 1 << node;

            q.push({node, mask});
            visited[node][mask] = true;
        }

        int steps = 0;

        while (!q.empty()) {
            int size = q.size();

            while (size--) {
                auto [node, mask] = q.front();
                q.pop();

                if (mask == fullMask) {
                    return steps;
                }

                for (int nei : graph[node]) {
                    int nextMask = mask | (1 << nei);
```

```

        if (!visited[nei][nextMask]) {
            visited[nei][nextMask] = true;
            q.push({nei, nextMask});
        }
    }
}

steps++;
}

return -1;
}
};

```

18.5 Complexity

Time:

$O(n * 2^n + \text{edges} * 2^n)$

Space:

$O(n * 2^n)$

19. Full Problem Walkthrough 8 — Traveling Salesman Problem

19.1 Problem Summary

Given distances between n cities.

Start at city 0.

Visit every city exactly once.

Return to city 0 with minimum cost.

19.2 State

$dp[mask][last]$ = minimum cost to start at 0, visit mask, and end at last

19.3 Transition

Move from last to next:

$dp[mask | (1 \ll next)][next] = \min(dp[...], dp[mask][last] + dist[last][next])$

19.4 C++ Code

```

int tsp(vector<vector<int>>& dist) {
    int n = dist.size();

    const int INF = 1e9;

```

```

vector<vector<int>> dp(1 << n, vector<int>(n, INF));

dp[1][0] = 0;

for (int mask = 0; mask < (1 << n); mask++) {
    for (int last = 0; last < n; last++) {
        if (dp[mask][last] == INF) {
            continue;
        }

        for (int next = 0; next < n; next++) {
            if ((mask & (1 << next)) == 0) {
                int nextMask = mask | (1 << next);

                dp[nextMask][next] = min(dp[nextMask][next],
                                          dp[mask][last] + dist[last][next]);
            }
        }
    }
}

int full = (1 << n) - 1;
int answer = INF;

for (int last = 1; last < n; last++) {
    answer = min(answer, dp[full][last] + dist[last][0]);
}

return answer;
}

```

19.5 Complexity

Time:

$O(n^2 * 2^n)$

Space:

$O(n * 2^n)$

20. Full Problem Walkthrough 9 — LeetCode 1434: Number of Ways to Wear Different Hats

20.1 Problem Summary

There are people and hats.

Each person has a list of hats they like.

Assign each person one unique hat they like.

Return number of ways.

20.2 Why Invert the DP?

People count is small, hats count can be 40.

Use mask over people.

Process hats one by one.

20.3 State

`dp[mask]` = number of ways to assign hats to people in mask

20.4 C++ Code

```
class Solution {
public:
    int numberWays(vector<vector<int>>& hats) {
        const int MOD = 1000000007;

        int people = hats.size();

        vector<vector<int>> hatToPeople(41);

        for (int p = 0; p < people; p++) {
            for (int hat : hats[p]) {
                hatToPeople[hat].push_back(p);
            }
        }

        vector<int> dp(1 << people, 0);
        dp[0] = 1;

        for (int hat = 1; hat <= 40; hat++) {
            vector<int> next = dp;

            for (int mask = 0; mask < (1 << people); mask++) {
                if (dp[mask] == 0) {
                    continue;
                }
                for (int p : hatToPeople[hat]) {
                    int newMask = mask | (1 << p);
                    next[newMask] = (next[newMask] + dp[mask]) % MOD;
                }
            }
            dp = next;
        }

        return dp[(1 << people) - 1];
    }
};
```

```

    }

    for (int p : hatToPeople[hat]) {
        if ((mask & (1 << p)) == 0) {
            int nextMask = mask | (1 << p);

            next[nextMask] = (next[nextMask] + dp[mask]) % MOD;
        }
    }

    dp = next;
}

return dp[(1 << people) - 1];
}
};

```

20.5 Complexity

Time:

$O(40 * 2^{\text{people}} * \text{averageLikes})$

Space:

$O(2^{\text{people}})$

21. Full Problem Walkthrough 10 — LeetCode 1125: Smallest Sufficient Team

21.1 Problem Summary

Given required skills and people with skills.
Return smallest team that covers all skills.

21.2 How to Identify Bitmask DP

Each skill can be a bit.
Each person covers a skill mask.
Need minimum-size set of people covering all bits.

21.3 State

$\text{dp}[\text{skillMask}]$ = vector of people forming smallest team for skillMask

21.4 C++ Code

```

class Solution {
public:
    vector<int> smallestSufficientTeam(vector<string>& req_skills,
                                     vector<vector<string>>& people) {
        int m = req_skills.size();

        unordered_map<string, int> skillIndex;

        for (int i = 0; i < m; i++) {
            skillIndex[req_skills[i]] = i;
        }

        int full = (1 << m) - 1;

        vector<vector<int>> dp(1 << m);

        for (int i = 0; i < people.size(); i++) {
            int personMask = 0;

            for (string& skill : people[i]) {
                if (skillIndex.count(skill)) {
                    personMask |= 1 << skillIndex[skill];
                }
            }

            if (personMask == 0) {
                continue;
            }

            vector<vector<int>> next = dp;

            for (int mask = 0; mask <= full; mask++) {
                if (mask != 0 && dp[mask].empty()) {
                    continue;
                }

                int combined = mask | personMask;

                if (combined == mask) {
                    continue;
                }
            }
        }
    }
};

```

```

        if (next[combined].empty() ||
            next[combined].size() > dp[mask].size() + 1) {
            next[combined] = dp[mask];
            next[combined].push_back(i);
        }
    }

    dp = next;
}

return dp[full];
}
};

```

21.5 Complexity

Time:

$O(\text{people} * 2^{\text{skills}} * \text{teamCopyCost})$

Space:

$O(2^{\text{skills}} * \text{teamSize})$

22. Full Problem Walkthrough 11 — Max Students Taking Exam

22.1 Problem Summary

Given seats grid with broken seats.

Students cannot sit adjacent left/right or diagonally upper-left/upper-right.

Return maximum students.

22.2 How to Identify Row Mask DP

Each row seating arrangement can be a mask.

Need ensure:

- mask uses only available seats
- no adjacent students in same row
- no diagonal conflict with previous row

22.3 State

$\text{dp}[\text{row}][\text{mask}] = \text{max students up to this row with current row mask}$

22.4 C++ Code

```

class Solution {
public:
    int maxStudents(vector<vector<char>>& seats) {
        int rows = seats.size();
        int cols = seats[0].size();
    }
};

```

```

vector<int> validSeats(rows, 0);

for (int r = 0; r < rows; r++) {
    int mask = 0;

    for (int c = 0; c < cols; c++) {
        if (seats[r][c] == '.') {
            mask |= 1 << c;
        }
    }

    validSeats[r] = mask;
}

vector<int> rowMasks;

for (int mask = 0; mask < (1 << cols); mask++) {
    if ((mask & (mask << 1)) == 0) {
        rowMasks.push_back(mask);
    }
}

vector<vector<int>> dp(rows, vector<int>(1 << cols, -1));

for (int mask : rowMasks) {
    if ((mask & validSeats[0]) == mask) {
        dp[0][mask] = __builtin_popcount(mask);
    }
}

for (int r = 1; r < rows; r++) {
    for (int mask : rowMasks) {
        if ((mask & validSeats[r]) != mask) {
            continue;
        }

        int count = __builtin_popcount(mask);

        for (int prev : rowMasks) {
            if (dp[r - 1][prev] == -1) {
                continue;
            }
        }
    }
}

```

```

    }

    if ((mask & (prev << 1)) == 0 &&
        (mask & (prev >> 1)) == 0) {
        dp[r][mask] = max(dp[r][mask],
                        dp[r - 1][prev] + count);
    }
}
}
}

int answer = 0;

for (int mask = 0; mask < (1 << cols); mask++) {
    answer = max(answer, dp[rows - 1][mask]);
}

return answer;
}
};

```

22.5 Complexity

Time:

$O(\text{rows} * \text{validMasks}^2)$

Space:

$O(\text{rows} * 2^{\text{cols}})$

23. Full Problem Walkthrough 12 — Minimum Incompatibility

23.1 Problem Summary

Partition nums into k groups of equal size.

Each group must have unique values.

Incompatibility of group:

$\text{max} - \text{min}$

Minimize total incompatibility.

23.2 How to Identify Bitmask DP

Need choose groups from unused elements.

n is small.

Precompute valid group masks.

23.3 State

$dp[mask]$ = minimum incompatibility for used elements mask

23.4 High-Level Algorithm

26. Group size:

$size = n / k$

27. Precompute all masks with size bits that have no duplicate values.

28. Store each group mask's incompatibility.

29. DP over masks by adding a valid disjoint group.

23.5 Complexity

Can be high but feasible for small n .

This is advanced bitmask group DP.

24. Full Problem Walkthrough 13 — Parallel Courses II

24.1 Problem Summary

There are n courses, prerequisites, and you can take at most k courses per semester.

Return minimum semesters.

24.2 How to Identify Bitmask DP

State is completed courses mask.

At each step, choose subset of available courses of size at most k .

24.3 State

$dp[mask]$ = minimum semesters needed to complete courses in mask

24.4 Key Idea

For each mask:

30. Find available courses whose prerequisites are satisfied.

31. Enumerate submasks of available courses.

32. Take up to k courses.

24.5 Lesson

Submask iteration is important when you must choose a subset of available options.

25. Full Problem Walkthrough 14 — Stickers to Spell Word

25.1 Problem Summary

Given stickers and target.

Each sticker has letters.

Return minimum stickers needed to form target.

25.2 Bitmask State

If target length is small, mask can represent which target characters are already covered.

$dp[mask]$ = minimum stickers needed to cover mask

Each sticker transitions to a larger mask.

25.3 Lesson

Bitmask can track individual characters, even if letters repeat.

26. Full Problem Walkthrough 15 — Campus Bikes II

26.1 Problem Summary

Assign each worker one bike.

Each bike used once.

Minimize total Manhattan distance.

26.2 Assignment DP

Workers are processed by index.

Mask represents used bikes.

26.3 State

$dp[mask]$ = minimum distance after assigning first $\text{popcount}(mask)$ workers

26.4 C++ Code

```
class Solution {
public:
    int assignBikes(vector<vector<int>>& workers,
                   vector<vector<int>>& bikes) {
        int w = workers.size();
        int b = bikes.size();

        const int INF = 1e9;

        vector<int> dp(1 << b, INF);
        dp[0] = 0;

        for (int mask = 0; mask < (1 << b); mask++) {
            int worker = __builtin_popcount(mask);

            if (worker >= w) {
                continue;
            }

            for (int bike = 0; bike < b; bike++) {
```

```

        if ((mask & (1 << bike)) == 0) {
            int dist = abs(workers[worker][0] - bikes[bike][0])
                + abs(workers[worker][1] - bikes[bike][1]);

            int nextMask = mask | (1 << bike);

            dp[nextMask] = min(dp[nextMask], dp[mask] + dist);
        }
    }
}

int answer = INF;

for (int mask = 0; mask < (1 << b); mask++) {
    if (__builtin_popcount(mask) == w) {
        answer = min(answer, dp[mask]);
    }
}

return answer;
}
};

```

26.5 Complexity

Time:

$O(\text{bikes} * 2^{\text{bikes}})$

Space:

$O(2^{\text{bikes}})$

27. Common Bitmask DP Edge Cases

27.1 $n = 0$

Full mask may be 0.

27.2 n Too Large

$1 \ll n$ overflows for large n .

Use:

$1LL \ll n$

when needed.

27.3 Duplicate Values

Bitmask tracks indices, not values.

Duplicates need special care.

27.4 Impossible State

Use INF or -1.

27.5 Empty Mask

Usually important base case:

```
dp[0] = 0
```

or:

```
dp[0] = 1
```

27.6 Full Mask

Usually answer is:

```
dp[(1 << n) - 1]
```

28. Common Bitmask DP Mistakes

Mistake 1 — Off-by-One Bit Index

If number is 1-based, use:

```
1 << (num - 1)
```

Mistake 2 — Forgetting Parentheses

Wrong:

```
mask & 1 << i
```

Better:

```
mask & (1 << i)
```

Mistake 3 — Overflow With $1 \ll n$

For $n \geq 31$, int overflows.

Use long long if needed.

Mistake 4 — Confusing Mask Meaning

Clearly define whether mask means:

- used items
- remaining items
- visited nodes
- covered skills

Mistake 5 — Not Checking INF State

Do not transition from impossible states.

29. Interview Explanation

Templates

29.1 Basic Bitmask DP

I use an integer mask to represent which items have already been selected. Each DP state corresponds to a subset, and transitions add one unused item.

29.2 Assignment DP

The number of set bits in the mask tells me how many workers/items have already been assigned. I assign the next worker to one unused option.

29.3 TSP DP

I define $dp[mask][last]$ as the minimum cost to visit all nodes in mask and end at last. Then I try moving to an unvisited next node.

29.4 Bitmask BFS

A normal BFS node is not enough because I need to know which nodes have been visited. So the BFS state is (currentNode, visitedMask).

29.5 Grouping DP

I precompute valid group masks, then use DP over used-element masks to add one valid group at a time.

30. Complexity Patterns

Generate subsets	$O(n * 2^n)$	output size
Basic $dp[mask]$	$O(n * 2^n)$	$O(2^n)$
Assignment DP	$O(n * 2^n)$	$O(2^n)$
TSP DP	$O(n^2 * 2^n)$	$O(n * 2^n)$
Bitmask BFS	$O((n + edges) * 2^n)$	$O(n * 2^n)$
Submask iteration	$O(3^n)$ total	varies
Row-mask DP	$O(rows * masks^2)$	$O(rows * masks)$
Grouping DP	$O(validGroups * 2^n)$	$O(2^n)$

31. Time-Boxed Practice Guidance

Intro Bitmask

Target:

30–45 minutes

Problems:

- Subsets
- Beautiful Arrangement
- Minimum XOR Sum
- Can I Win

Medium Bitmask

Target:

45–75 minutes

Problems:

- Partition to K Equal Sum Subsets
- Shortest Path Visiting All Nodes
- Campus Bikes II
- Smallest Sufficient Team

Hard Bitmask

Target:

75–120 minutes

Problems:

- Number of Ways to Wear Different Hats
 - Max Students Taking Exam
 - Minimum Incompatibility
 - Parallel Courses II
 - Stickers to Spell Word
-

32. Expanded Practice Set

32.1 Must-Do Medium

33. Subsets
34. Beautiful Arrangement
35. Minimum XOR Sum of Two Arrays
36. Can I Win
37. Partition to K Equal Sum Subsets
38. Matchsticks to Square
39. Campus Bikes II
40. Shortest Path Visiting All Nodes

32.2 Strong Interview Practice

41. Smallest Sufficient Team
42. Number of Ways to Wear Different Hats
43. Max Students Taking Exam
44. Parallel Courses II
45. Stickers to Spell Word
46. Minimum Incompatibility
47. Fair Distribution of Cookies

32.3 Hard / Stretch

48. Minimum Incompatibility
 49. Parallel Courses II
 50. Stickers to Spell Word
 51. Number of Ways to Wear Different Hats
 52. Max Students Taking Exam
 53. Special Permutations
 54. Count Number of Maximum Bitwise-OR Subsets
 55. Find the Shortest Superstring
-

33. Suggested Study Order

Step 1 — Bit Basics

Start with:

- 56. Subsets
- 57. Beautiful Arrangement

Goal:

Learn mask representation and transitions.

Step 2 — Assignment DP

Solve:

- 58. Minimum XOR Sum
- 59. Campus Bikes II

Goal:

Learn popcount(mask) as assigned index.

Step 3 — Game / Used Number DP

Solve:

- 60. Can I Win

Goal:

Learn memoized recursion over masks.

Step 4 — Grouping DP

Solve:

- 61. Partition to K Equal Sum Subsets
- 62. Matchsticks to Square

Goal:

Learn bucket modulo and used-mask states.

Step 5 — Graph + Bitmask

Solve:

- 63. Shortest Path Visiting All Nodes
- 64. TSP

Goal:

Learn (node, mask) state.

Step 6 — Advanced Subset DP

Solve:

- 65. Smallest Sufficient Team
- 66. Hats
- 67. Max Students
- 68. Parallel Courses II

Goal:

Learn masks for skills, people, rows, and courses.

34. Pattern Decision Table

choose subset	mask	Subsets

Problem Clue	Bitmask State	Example
assign one-to-one	dp[mask]	Minimum XOR Sum
used numbers game	memo[mask]	Can I Win
visit all nodes	(node, mask)	Shortest Path Visiting All Nodes
TSP	dp[mask][last]	Traveling Salesman
cover skills	skillMask	Smallest Team
assign hats to people	personMask	Hats
row seating	rowMask	Max Students
choose courses	courseMask	Parallel Courses
cover target letters	charMask	Stickers

35. Mini Problem 1 — Count Maximum Bitwise OR Subsets

Given nums, count subsets whose OR equals maximum possible OR.
Backtracking is common, but bitmask enumeration works:

```
class Solution {
public:
    int countMaxOrSubsets(vector<int>& nums) {
        int n = nums.size();

        int maxOr = 0;

        for (int x : nums) {
            maxOr |= x;
        }

        int count = 0;

        for (int mask = 0; mask < (1 << n); mask++) {
            int value = 0;

            for (int i = 0; i < n; i++) {
                if (mask & (1 << i)) {
```

```

        value |= nums[i];
    }
}

if (value == maxOr) {
    count++;
}

return count;
};

```

36. Mini Problem 2 — Iterate Submasks Example

```

void printSubmasks(int mask) {
    for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
        cout << sub << endl;
    }
}

```

For mask = 1011, this iterates all non-empty submasks of 1011.

37. Mini Problem 3 — Find First Missing Skill Mask

```
int missing = fullMask ^ currentMask;
```

or:

```
int missing = fullMask & ~currentMask;
```

Used when determining uncovered requirements.

38. Mini Problem 4 — Remove Lowest Set Bit

```
int lowbit = mask & -mask;
int newMask = mask ^ lowbit;
```

Useful for iterating set bits efficiently.

39. Common Edge Case Checklist

Before submitting bitmask DP, test:

n = 0
n = 1
full mask
empty mask
duplicate values
all items already used
no valid transition
impossible target
graph with one node
all skills covered by one person
no person has useful skill

40. Debugging Bitmask DP

If bitmask DP fails:

69. Define exactly what mask represents.
 70. Check whether bits are 0-based or 1-based.
 71. Verify full mask.
 72. Verify base state.
 73. Check transition only uses valid masks.
 74. Check impossible-state handling.
 75. Check popcount(mask) meaning.
 76. Check overflow of $1 \ll n$.
 77. Print masks in binary for small cases.
-

41. What to Master Before Chapter 41

Before moving to Tree Dynamic Programming, make sure you can:

- Represent a subset with a mask
- Check, set, clear, and toggle bits
- Iterate masks
- Iterate items inside a mask
- Iterate submasks
- Define `dp[mask]`
- Define `dp[mask][last]`
- Solve assignment DP
- Solve Can I Win
- Solve Shortest Path Visiting All Nodes
- Understand bitmask BFS

- Understand grouping with masks
 - Recognize when n is small enough for 2^n
-

42. Final Cheat Sheet

Full Mask

```
int full = (1 << n) - 1;
```

Check Bit

```
if (mask & (1 << i)) {  
}
```

Set Bit

```
mask | (1 << i)
```

Clear Bit

```
mask & ~(1 << i)
```

Iterate Masks

```
for (int mask = 0; mask < (1 << n); mask++) {  
}
```

Iterate Unused Items

```
for (int i = 0; i < n; i++) {  
    if ((mask & (1 << i)) == 0) {  
        int nextMask = mask | (1 << i);  
    }  
}
```

Iterate Submasks

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {  
}
```

Assignment DP

```
int i = __builtin_popcount(mask);  
dp[nextMask] = min(dp[nextMask], dp[mask] + cost(i, chosen));
```

TSP DP

```
dp[mask | (1 << next)][next] =  
    min(dp[mask | (1 << next)][next],  
        dp[mask][last] + dist[last][next]);
```

43. Core Takeaway

Bitmask DP is subset DP.

Use it when:

n is small
you need to remember which items are used

The mask is the memory.

The most common states are:

dp[mask]
dp[mask][last]
visited[node][mask]

The key skills are:

- bit operations
- popcount
- mask transitions
- submask iteration
- recognizing when 2^n is feasible

Bitmask DP turns many factorial brute-force problems into:

$O(n * 2^n)$

or:

$O(n^2 * 2^n)$

which is often acceptable for small n.
